SLIIT
Discover Your Future

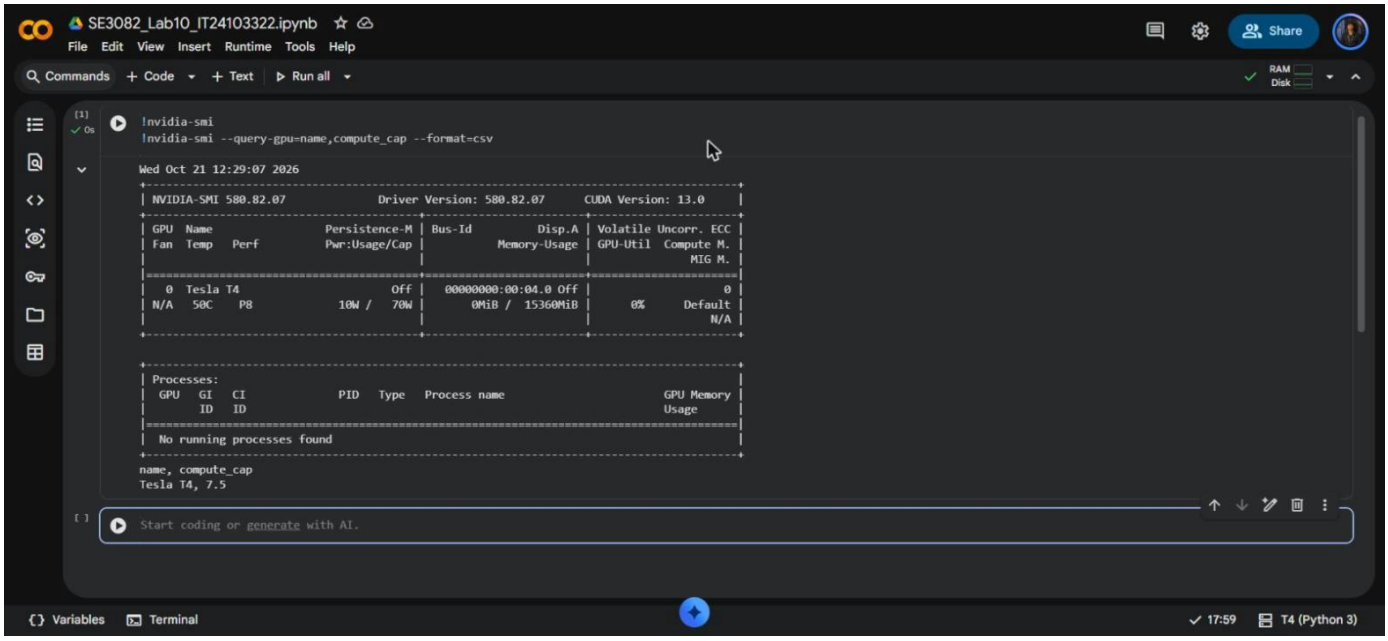
BSc (Hons) in Computer Science
Year 3

Lab Sheets

SE3082 – Parallel Computing **Semster 1, 2026**

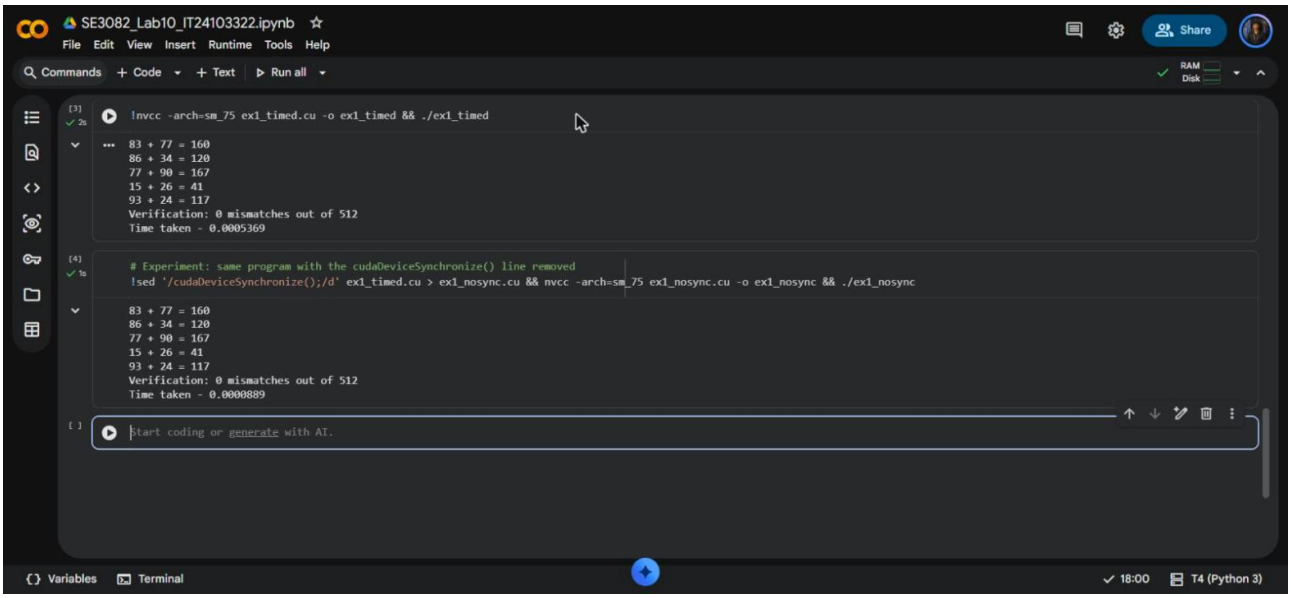
IT24103322

Exercise 1



A new notebook SE3082_Lab10_IT24103322 was created in Colab with the T4 GPU runtime. nvidia-smi shows a Tesla T4 with compute capability 7.5, so every program was compiled with -arch=sm_75.

a. **Compile and run the timed program.**

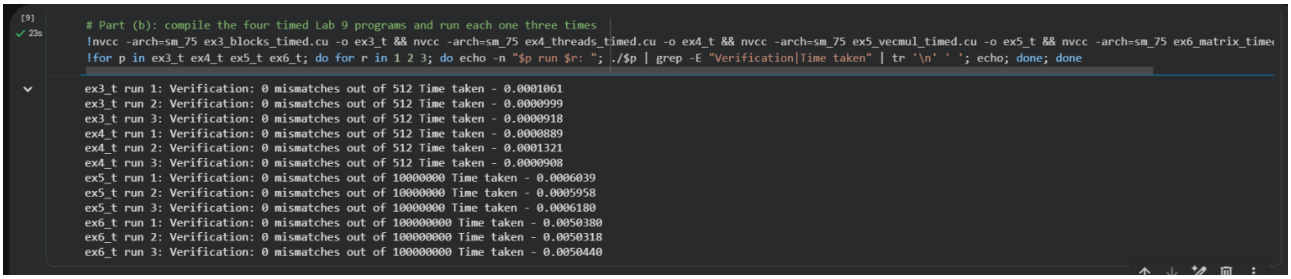


The timer uses cpuSecond() (gettimeofday) around the kernel launch, with cudaDeviceSynchronize() before the clock is stopped. The first five results are 83 + 77 = 160 … 93 + 24 = 117, Verification: 0 mismatches out of 512, and Time taken - 0.0005369 s.

Experiment – remove cudaDeviceSynchronize(). What happens to the time? Does the answer still verify?

The second cell copies the program without the cudaDeviceSynchronize() line. The time dropped from 0.0005369 s to 0.0000889 s. A kernel launch is asynchronous: the CPU starts the GPU and carries on straight away, so without the sync only the launch is timed, not the work. The answer still verifies (0 mismatches) because the cudaMemcpy back to the host waits for the kernel to finish before it copies.

b. **Add the timer to the Lab 9 programs (Exercises 3, 4, 5 and 6) and record their times.**



Lab 9 program	Launch configuration	Run 1 (s)	Run 2 (s)	Run 3 (s)
Exercise 3	512 blocks × 1 thread	0.0001061	0.0000999	0.0000918
Exercise 4	1 block × 512 threads	0.0000889	0.0001321	0.0000908
Exercise 5	19,532 blocks × 512 threads	0.0006039	0.0005958	0.0006180
Exercise 6	313 × 625 blocks of 32 × 16	0.0050380	0.0050318	0.0050440

Every run reported 0 mismatches.

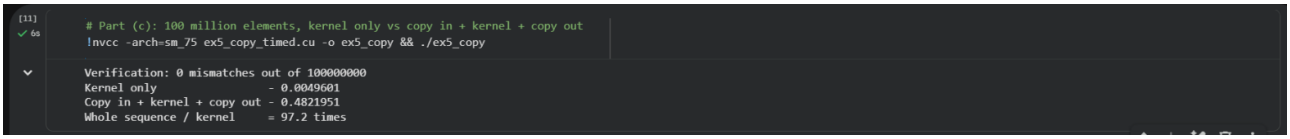
1. **512 blocks of one thread or one block of 512 threads – which is faster, and why?**

   One block of 512 threads should be faster, because its threads fill 16 complete warps, while 512 one-thread blocks use 512 warps with 31 of 32 lanes idle. In the measurements both took about 0.1 ms and the order changed between runs: with only 512 additions the launch overhead is larger than the work, so the difference is hidden in the noise.

2. **Exercise 6 has 10 times as many elements as Exercise 5. Does it take 10 times as long?**

   Close to it: about 5.04 ms against 0.60 ms, which is 8.4 times. Both kernels read two numbers and write one per element, so the time grows with the amount of data. It is a little under 10 times because part of the Exercise 5 time is the fixed cost of the launch.

c. **Kernel time versus copy time for 100 million elements.**



   With N = 100,000,000 (about 400 MB per array) the kernel alone took 0.0049601 s, while copy in + kernel + copy out took 0.4821951 s, about 97 times longer. The 1.2 GB of data has to travel over the PCIe bus, which is much slower than the GPU's own memory. So the GPU is only worth using for work that does a lot of calculation per number copied, or when the data can stay on the GPU for many kernels; a single multiply per element is mostly copy time.